INTERVIEW ROUND II

APPLICATION DEVELOPMENT ASSIGNMENT

Economic Indicators Dashboard

Design reference: Economic Indicator Assignment — Figma design (frame 1-2)

Evaluation viewport: 1214 × 2159 CSS pixels

Required application: React frontend and FastAPI backend

Assignment. Build and deploy the complete dashboard shown in the supplied design. The work includes the user interface, backend API, collection and presentation of dashboard data, database access, authentication, and deployment. The interview panel will review the working application and repository and will ask you to explain the approach you took.

I. Purpose and scope

Reproduce the supplied economic indicators screen as a working application. The Figma file is the design authority for layout, typography, colours, charts, cards, labels and visible interactions. The supplied image is included at the end of this assignment as a visual reference; use the Figma frame for detailed measurements and behaviour.

The screen is to be evaluated at 1214 × 2159 CSS pixels. At this size, the complete intended screen should be legible and stable, without unintended horizontal scrolling, clipped content or disruptive layout shifts.

The visible dashboard comprises a live market ticker and six indicator panels:

- GDP Growth — India and World, with year-on-year values and historical comparison;
- Debt to GDP Ratio — India, with the comparison and trend shown in the design;
- Imports, Exports and Trade Balance;
- GST Tax Collection, including the monthly average and historical comparison;
- Inflation — India, including the RBI target and historical trend; and
- Government Expenditure, including total and capital expenditure.

Reproduce the charts, period labels, summary figures, changes from the comparison period and other values shown in the design. Make each visible control operate as represented. Where the design leaves behaviour open, use a reasonable choice and record it in the README.

II. Data and application requirements

Use publicly accessible NSE endpoints or pages as the principal source for data available from NSE. The browser must request dashboard data from your FastAPI backend; it must not scrape NSE directly. Where NSE does not publish a required indicator, a clearly identified public source may be used. In every case, document the source, observation date, any transformation and any limitation.

- Display the source and “as of” date for each indicator or group of indicators. Identify non-NSE sources in the interface and README.
- Validate responses before storing or displaying them. Use timeouts, caching and rate-aware refreshes. Handle missing values and upstream failures without inventing figures or replacing the last valid observation with invalid data.
- Provide useful loading, empty, error, stale and unavailable states. Label retained data as stale when a refresh fails; show missing values as “—”.
- Use a server-side scheduled collector or equivalent process for upstream requests. Explain refresh frequency and how the frontend receives updates; SSE or a documented equivalent may be used.
- Provide an authentication flow. JWT authentication is preferred. Protect routes that should require sign-in and document token handling, expiry and the method for granting the panel evaluation access.

III. Technical and security conditions

- Build the frontend in React and the API in FastAPI. Keep the API base URL configurable and do not place upstream credentials in browser code.
- Use a database for snapshots or history where required. An integrated database may run on the API server. Direct, parameterized SQL queries are preferred; an ORM is not required. Never concatenate untrusted input into SQL.
- The database must not be publicly reachable. Permit connections only from the backend machine or its private network identity, enforced through the database bind address and appropriate firewall or security-group rules.
- If a proxy is placed in front of the database, it must be a private TCP proxy or equivalent control that accepts connections only from the backend server. A public HTTP reverse proxy is not a database security boundary.
- Deploy the frontend and API on separate websites and on different machines or hosting environments. Use HTTPS for both and restrict API CORS to the deployed frontend origin.
- Provide a health-check route and document the API routes, request/response formats, authentication requirements, environment variable names, database setup and local run instructions.

IV. Material to be submitted

- The source repository, including frontend and backend code, database schema or migrations, and a README.
- The live frontend URL, API URL or API documentation URL, and health-check URL. Ensure the interview panel can access them during evaluation.
- A portfolio website URL and links to your flagship projects. For each selected project, briefly state your role, the principal technologies used and what you built.
- A test account or evaluation instructions, shared separately from public source code. Do not commit passwords, signing keys, tokens or production credentials.
- A short list of assumptions, known limitations and any public fallback sources used.

V. Use of AI and project presentation

The use of AI tools is permitted. You remain responsible for the correctness, security and maintainability of the submitted application and must be able to explain and defend the work, including code produced or revised with AI assistance.

After the assessment, submit a screen-recorded video in which you demonstrate the application and explain the work in your own words. The presentation should cover:

- The complete user flow through the application and how the frontend, API, data collection and database work together;
- How you built the application, the main challenges you faced and how you addressed them;
- The important technical and design decisions you made and the reasons for those decisions;
- If you used AI: which tasks you used it for, how you incorporated its output, why AI was appropriate for those tasks, and how you checked the result;
- If you did not use AI: which tasks you chose to complete without it and why AI was not the right choice for those tasks.

The video is intended to help the panel understand your process and judgment. It is not a promotional video. You should be able to explain the complete application, including work delegated to tools.

VI. Assessment

The panel will award marks for design fidelity (25), application behaviour (20), backend and data handling (15), security and database access (15), performance and code quality (15), and deployment and documentation (10), for a total of 100 marks. Assessment will consider accuracy to the Figma design,

working interactions and data, sound engineering and security, performance at the required screen size, and the quality of deployment and documentation.

APPENDIX — DASHBOARD REFERENCE

The image below is provided for context. The linked Figma file remains the design reference for implementation.

